Restaurant POS as a Service

High Level Design (HLD) - Backend

Multi-tenant SaaS for hotels and restaurants, in the style of Petpooja. Table-wise orders, automatic KOT, GST billing, role-based access, monthly subscription.

Item	Decision
Business model	Each hotel is a tenant. Subscription Rs 1000 per month (plan limits and GST treatment to be finalised).
Backend stack	NestJS (TypeScript), PostgreSQL with Row Level Security, Redis, BullMQ, Socket.IO
Architecture	Modular monolith first, with clear module boundaries so heavy parts can be split out later
Hosting	AWS ap-south-1 (Mumbai) or equivalent, containers, managed Postgres and Redis
Status	Design plus a generated backend folder skeleton (pos-backend.zip)

1. Goals and scope

You sell the software to hotels. A hotel pays a monthly subscription and then uses it for daily operations: manage tables, take orders, send items to the right kitchen section as a KOT, print the bill with GST, collect payment, and see reports.

Core promise to a hotel

- Table-wise ordering. Open Table 1, add Paneer, Roti, Chicken, and the order belongs to that table.
- Automatic KOT. Items are routed to their station (Kitchen, Tandoor, Bar) and a KOT is created and printed without extra clicks.
- Easy bill. When the table is done, one tap generates a GST bill with discount, service charge, round-off. Payment by cash, card, UPI or split.
- Food photos and prices in the menu, per-item tax group.
- Role-based access. Owner, Manager, Cashier, Waiter, Kitchen, Accountant each see and do only what they should.
- Subscription control. If a hotel does not pay, the system moves to a grace period and then read-only, without deleting data.

Out of scope for version 1

- Online aggregator integrations (Zomato, Swiggy), full accounting, payroll, hotel room booking. Design keeps room for them as separate modules later.

2. Users and role-based access (RBAC)

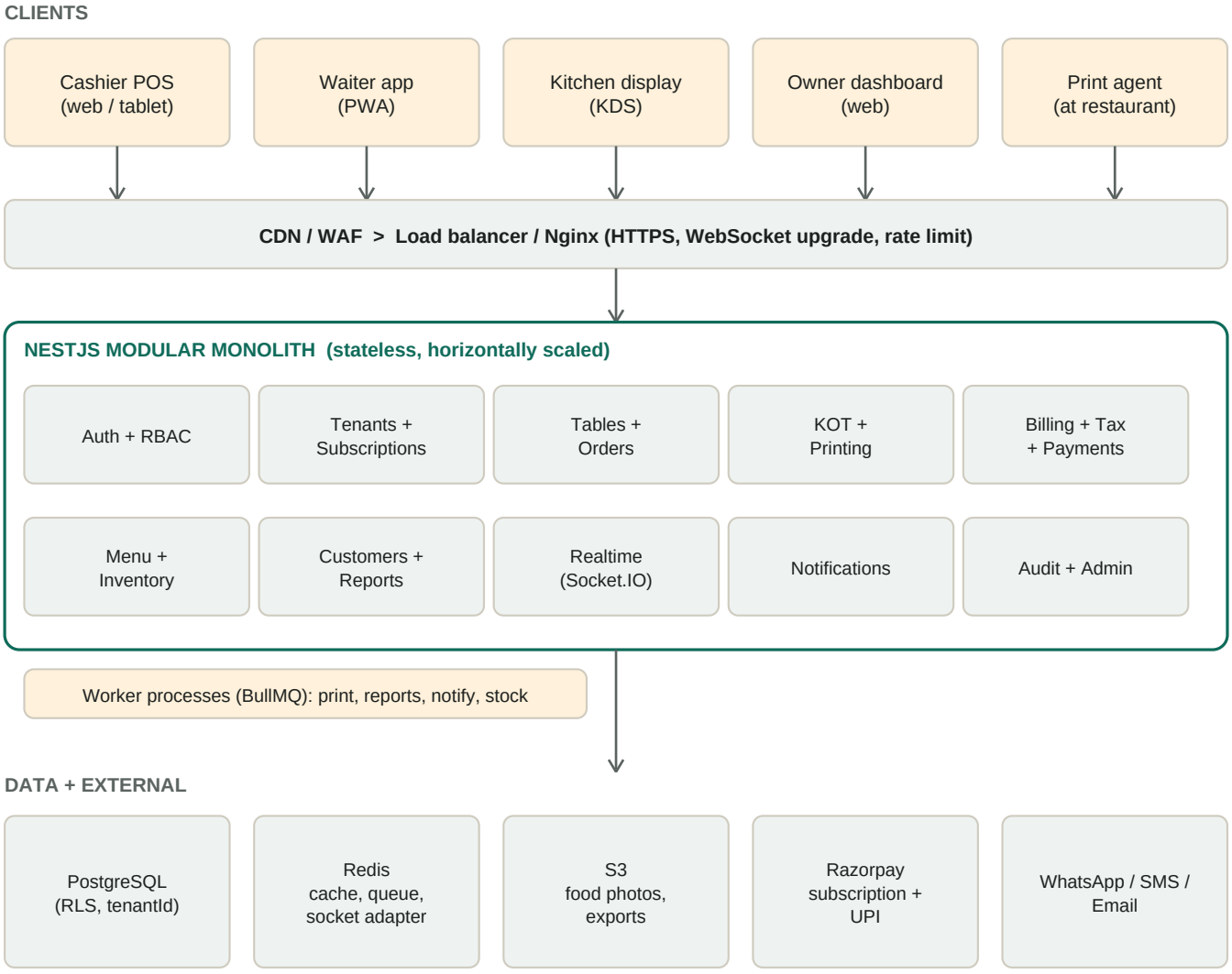
Authorization is permission-based. A permission is a key such as **bill.discount**. A role is a bundle of permissions. System roles are seeded for every new tenant, and an owner can create custom roles. Controllers declare what they need using a decorator, and a global PermissionsGuard enforces it. Roles are never hard-coded inside business logic.

Capability (permission key)	Owner	Manager	Cashier	Waiter	Kitchen	Accountant
Create / edit order (order.create)	Yes	Yes	Yes	Yes	-	-
Send KOT (kot.send)	Yes	Yes	Yes	Yes	-	-
Update KOT status (kot.update)	Yes	Yes	-	-	Yes	-
Cancel item after KOT (order.cancel_item)	Yes	Yes	Limited (needs PIN)	-	-	-
Apply discount (bill.discount)	Yes	Yes	Limited (max %)	-	-	-
Generate bill (bill.generate)	Yes	Yes	Yes	Limited	-	-
Settle payment (payment.create)	Yes	Yes	Yes	-	-	-
Void bill / refund (bill.void)	Yes	Yes	-	-	-	-
Edit menu and prices (menu.manage)	Yes	Yes	-	-	-	-
View reports (report.view)	Yes	Yes	Limited	-	-	Yes
Manage staff and roles (user.manage)	Yes	Limited	-	-	-	-
Manage subscription (subscription.manage)	Yes	-	-	-	-	-

Two more levels sit on top of tenant roles: a **Super Admin** role for you, the platform owner (list tenants, change plans, suspend, support impersonation with audit), and **outlet scoping**, where a user can be limited to one outlet of a multi-outlet hotel.

3. System architecture

One NestJS codebase is deployed as two process types: the **API** (HTTP and WebSocket) and the **Worker** (queue jobs). Both are stateless, so more copies can be added behind the load balancer. State lives in PostgreSQL, Redis and S3.



Why a modular monolith first

- One deployable, one database transaction across order, KOT and bill. This matters because "send KOT" must either fully succeed or fully fail.
- Each folder under **src/modules** owns its tables and exposes a service interface. Other modules call the service, never its tables. That makes later extraction possible (printing, notifications and reports are the first candidates).
- Much cheaper to run and debug for a small team than microservices.

4. Multi-tenancy design

Option	Isolation	Cost / ops	Decision
Shared schema, tenantId column + RLS	Logical, enforced by Postgres policy	Lowest. One migration for all tenants	**Chosen for v1**
Schema per tenant	Stronger	Migrations multiply by tenant count	No
Database per tenant	Strongest	Very high at 1000+ tenants	Only for large enterprise customers later

How isolation is enforced (three layers)

- **Token:** the JWT carries tenantId, outletId, roleId. The client can never choose the tenant.
- **Application:** a tenant context (AsyncLocalStorage) is set by middleware. The Prisma extension adds tenantId to every query and runs **SET LOCAL app.tenant_id** inside each transaction.
- **Database:** Row Level Security policies on every tenant table, so even a forgotten WHERE clause cannot return another hotel's data. An automated e2e test (tenant-isolation) proves tenant A cannot read tenant B.
- Plan limits (outlets, users) are checked by PlanLimitGuard. Tenant status and subscription are cached in Redis and invalidated on change.

5. Core flow: Table 1 order to receipt

Example: Table 1 orders Paneer, Roti and Chicken. Paneer and Roti are cooked in the main Kitchen, Chicken in the Tandoor.

#	Step	What the backend does
1	Waiter logs in with PIN	POST /auth/pin-login. Returns short-lived access token with permission list.
2	Open Table 1	POST /tables/{id}/orders. Creates Order (OPEN), table status becomes RUNNING. Order number from an atomic per-outlet sequence.
3	Add items	POST /orders/{id}/items. Each line stores a snapshot of name and price at that moment, so later menu price changes never alter old orders.
4	Send KOT	POST /orders/{id}/kot, one DB transaction. KOT router groups unsent items by station: KOT-A (Kitchen: Paneer, Roti), KOT-B (Tandoor: Chicken). KOT numbers assigned. Order becomes KOT_SENT. Domain event **kot.created** is emitted after commit.
5	Print and display	Event listener enqueues print jobs (BullMQ) per station printer and pushes the KOT over WebSocket to room station:{id}. The local print agent or kitchen screen receives it within a second.
6	Kitchen progress	Kitchen taps NEW > PREPARING > READY. Waiter app gets a realtime update, table card shows it.
7	Customer adds more	Add items again, send KOT again. Only items not yet sent create a new KOT. Earlier KOTs stay untouched.
8	Generate bill	POST /orders/{id}/bill. BillCalculator computes subtotal, discount, service charge, CGST+SGST per tax group, round-off. Continuous GST bill number is allocated. Table becomes BILLED. Receipt is printed.
9	Take payment	POST /bills/{id}/payments (cash, card, UPI, split allowed). When paid amount is at least the total, Bill and Order become PAID and the table becomes FREE. Idempotency-Key prevents double charge.
10	After commit (async)	Worker deducts stock by recipe, sends WhatsApp or SMS receipt, writes audit log, updates report aggregates.

State machines

Entity	States and transitions
Table	FREE > RUNNING (order opened) > BILLED (bill generated) > FREE (paid). RESERVED can go to RUNNING.
Order	OPEN > KOT_SENT > SERVED > BILLED > PAID. OPEN or KOT_SENT can go to CANCELLED (permission and reason required).
KOT	NEW > PREPARING > READY > SERVED. NEW or PREPARING can go to CANCELLED.
Bill	DRAFT > GENERATED > PAID. GENERATED or PAID can go to VOID (permission, reason and audit required). A bill number is never reused.

Entity	States and transitions
Subscription	TRIALING > ACTIVE > PAST_DUE (grace) > EXPIRED. ACTIVE can go to CANCELLED.

6. Billing and tax rules

- All money is stored as **integer paise**. Tax rates in basis points (250 = 2.5%). No floating point anywhere.
- Each menu item has a TaxGroup (for example GST 5% = CGST 2.5% + SGST 2.5%, SAC 996331). The bill stores taxable value and tax per line, plus a SAC-wise tax summary.
- Order of calculation: items total, minus discount, plus service charge (optional, per tenant setting), then tax, then round-off to nearest rupee.
- Inclusive or exclusive pricing is a tenant setting. Interstate IGST is supported in the schema.
- Bill numbers are continuous per outlet and financial year, allocated with a database sequence row lock. Voided bills keep their number.
- **Important:** GST rates and rules differ by restaurant type and change over time. Keep them as configurable data and have a CA confirm the defaults before launch.

7. Data model (key entities)

The full starting schema is in **prisma/schema.prisma** inside the zip. Every tenant-owned table has tenantId and an index starting with tenantId.

Group	Entities	Notes
Platform	Plan, Tenant, Subscription	Plan holds price in paise and limits. One subscription per tenant.
Access	User, Role, Permission, RolePermission	Permission keys are global, roles are per tenant.
Setup	Outlet, Area, DiningTable, Station	Station links menu items to kitchen, tandoor or bar printers.
Menu	MenuCategory, MenuItem, TaxGroup	Item has price, veg flag, image key, station, tax group.
Order	Order, OrderItem, Kot, KotItem	OrderItem keeps name and price snapshot. KotItem links to OrderItem.
Billing	Bill, BillItem, Payment	A bill can have many payments (split). Totals stored, not recomputed.
Support	AuditLog, PrintJob, StockLedger, Customer, Shift	Append-only where possible (audit, stock ledger).

Data rules

- Money is integer paise. Dates are stored in UTC and shown in IST. Reports use a business-day cut-off (for example 4 AM) so late-night sales count in the right day.
- Soft delete for menu, users and tables. Hard delete never for orders, bills, payments and audit.
- Large append-heavy tables (Order, Bill, AuditLog) are candidates for monthly partitioning once volume grows.

8. API surface

REST over HTTPS, versioned under **/v1**, OpenAPI generated by Swagger. Writes that must not be repeated accept an **Idempotency-Key** header. Responses use one envelope and one error format.

Area	Main endpoints
Auth	POST /auth/login, /auth/pin-login, /auth/refresh, /auth/logout
Tenant and setup	GET/PATCH /tenant, /outlets, /areas, /tables, /stations
Staff and roles	/users, /roles, /roles/{id}/permissions
Menu	/menu/categories, /menu/items, /menu/items/{id}/image-upload-url
Orders	POST /tables/{id}/orders, POST /orders/{id}/items, PATCH /orders/{id}/items/{itemId}, POST /orders/{id}/transfer
KOT	POST /orders/{id}/kot, GET /kots?station=, PATCH /kots/{id}/status, POST /kots/{id}/reprint
Billing	POST /orders/{id}/bill, PATCH /bills/{id}/discount, POST /bills/{id}/split, POST /bills/{id}/void
Payments	POST /bills/{id}/payments, POST /bills/{id}/refund
Reports	GET /reports/sales, /reports/items, /reports/tax, /reports/staff, POST /reports/export
Subscription	GET /subscription, POST /subscription/subscribe, POST /webhooks/razorpay
Super admin	/admin/tenants, /admin/plans, /admin/metrics, POST /admin/tenants/{id}/suspend

9. Realtime and printing

Realtime (Socket.IO with Redis adapter)

- On connect the socket authenticates with the same JWT, then joins rooms: **tenant:{id}**, **outlet:{id}**, **station:{id}**. A kitchen screen only hears its own station.
- Events: table.updated, kot.created, kot.status, bill.paid. The Redis adapter lets any API pod broadcast to sockets connected to another pod.
- REST remains the source of truth. After reconnect the client re-fetches state, so a missed event never causes wrong data.

Printing

- Cloud cannot reach a printer inside a restaurant LAN directly. A small **Print Agent** runs on a restaurant PC or Android device, connects out to the backend over WebSocket, and prints ESC/POS to the local thermal printer (58 or 80 mm).
- Backend creates PrintJob rows and queues them. The agent acknowledges each job. Failed jobs retry, and the cashier screen shows a clear "printer offline" status with a reprint button.
- Fallback: browser print of the same template if no agent is installed.

Restaurant internet drops (offline tolerance)

- The POS client is a PWA that queues actions locally (IndexedDB) with Idempotency-Keys and syncs when back online, so retries are safe.
- KOT printing keeps working on LAN through the agent if the agent caches the last KOT templates. Full offline billing is a later phase.

10. Subscription and tenant lifecycle

Stage	What happens	Product behaviour
Signup	TenantOnboarding creates tenant, owner user, default roles, default tax groups and a trial subscription in one transaction.	Full access during trial.
Active	Razorpay recurring payment each month (Rs 1000 plan). Webhook (signature verified) marks period paid.	Full access.
Payment failed	Status PAST_DUE. Reminders by WhatsApp, SMS and email. Grace period (for example 5 days).	Full access with banner.
Expired	Daily job sets EXPIRED after grace.	**Read-only**: can view and print old bills, cannot create orders. SubscriptionGuard enforces this.
Cancelled	Owner cancels or never pays.	Data kept for a retention period, then exported and archived per policy.

Plans are rows in the Plan table (price, maxOutlets, maxUsers, feature flags). Starting with one plan at Rs 1000 per month is fine. Add a higher plan for multi-outlet chains later without code changes. Decide whether the price is inclusive of GST and issue platform GST invoices to your customers (subscription-invoice service).

11. Security

- Passwords hashed with argon2. Staff PIN hashed too, with strict rate limit and lockout.
- Access token 15 minutes, refresh token rotation with the hash stored in Redis, revocable per device.
- Global guards order: JwtAuthGuard, TenantGuard, SubscriptionGuard, PermissionsGuard. Public routes opt out explicitly with @Public().
- Sensitive actions (discount above limit, cancel after KOT, void bill, price change, role change) require permission and always write an AuditLog entry with user, time and reason.
- Input validation with class-validator on every DTO. Helmet, CORS allow-list, throttling per IP and per tenant.

- TLS everywhere. Secrets in a secrets manager, never in the repo. Encrypted database and S3. Webhook signatures verified. Presigned URLs for uploads.
- Customer phone numbers are personal data: collect with purpose, allow deletion, follow India's data protection law requirements.

12. Scale plan

Rough sizing (assumption, to be validated): 1000 hotels, about 300 orders per hotel per day gives 300k orders a day, around 3.5 per second on average and perhaps 50 to 100 per second at dinner peak across all tenants. This fits comfortably on one well-tuned PostgreSQL primary, so the first priority is correctness and good indexes, not exotic architecture.

Phase	Tenants	Changes
1 - Launch	0 to 200	2 API pods + 1 worker, managed Postgres (Multi-AZ), Redis, S3. Daily backups, point-in-time recovery.
2 - Growth	200 to 2000	Read replica for reports. PgBouncer connection pooling. Monthly partitions on Order, Bill, AuditLog. Autoscaling on CPU and queue depth. Dedicated worker pools per queue.
3 - Scale	2000+	Extract Reports, Printing and Notifications into services. Tenant-to-shard map to spread tenants across several database clusters. Analytics store for heavy reports. Multi-region read for DR.

Performance rules from day one

- Every index starts with tenantId. Hot lists (open orders per outlet, KOTs per station) are indexed by status.
- Reports never scan raw orders for long ranges. Maintain daily aggregate tables updated by the worker.
- Cache menu and tax groups per tenant in Redis. Invalidate on menu change.
- Keep request handlers short. Printing, WhatsApp, stock deduction and exports always go through queues.

13. Reliability and observability

- Health endpoints for liveness and readiness. Graceful shutdown so in-flight orders finish.
- Structured logs with requestId, tenantId, userId. Metrics: request latency, queue depth, print failure rate, KOT-to-ready time, active sockets per tenant.
- Error tracking (Sentry) and alerts on payment webhook failures and queue backlogs.
- Backups with tested restore. Target: RPO 5 minutes, RTO 1 hour (to be confirmed).
- CI pipeline: lint, unit tests (bill calculator is critical), e2e (order to bill, tenant isolation), build and push image.

14. Backend folder structure

This is the exact structure generated in **pos-backend.zip**. Each module folder follows the same shape: module, controller, service, dto, plus feature-specific services. The shared building blocks live in **common** and **database**.

Folder	Responsibility
src/config	Typed configuration and environment validation
src/common	Decorators, guards (auth, tenant, subscription, permissions), interceptors, filters, tenant context, money and tax utilities
src/database	Prisma service and the extension that sets tenant for RLS
src/modules	One folder per business area (auth, tenants, subscriptions, tables, menu, orders, kot, billing, payments, tax, printing, inventory, reports, ...)
src/events	Domain event listeners (order placed, bill settled, subscription changed)
src/jobs	BullMQ processors run by the worker process
prisma	schema.prisma, migrations, seed, rls.sql
test	Unit and e2e tests, including tenant isolation test
infra, docs, scripts	Nginx, Kubernetes manifests, ADRs, OpenAPI, seed and admin scripts

```
pos-backend/
|-- .github/
|   `-- workflows/
|       `-- ci.yml
|-- docs/
|   |-- adr/
|   |   |-- 0001-multi-tenancy-shared-schema-rls.md
|   |   `-- 0002-nestjs-modular-monolith.md
|   |-- api/
|   |   `-- openapi.yaml
|   |-- ERD.md
|   `-- HLD.md
|-- infra/
|   |-- k8s/
|   |   |-- deployment.yaml
|   |   |-- hpa.yaml
|   |   |-- service.yaml
|   |   `-- worker-deployment.yaml
|   |-- nginx/
|   |   `-- nginx.conf
|   `-- terraform/
|-- prisma/
|   |-- migrations/
|   |-- rls.sql
|   |-- schema.prisma
|   `-- seed.ts
|-- scripts/
|   |-- create-super-admin.ts
|   `-- seed-demo-tenant.ts
|-- src/
|   |-- common/
|   |   |-- constants/
|   |   |   |-- events.ts
|   |   |   |-- permissions.ts
|   |   |   |-- queue-names.ts
|   |   |   `-- roles.ts
|   |   |-- context/
|   |   |   `-- tenant-context.ts
|   |   |-- decorators/
|   |   |   |-- current-tenant.decorator.ts
|   |   |   |-- current-user.decorator.ts
|   |   |   |-- permissions.decorator.ts
|   |   |   |-- public.decorator.ts
|   |   |   `-- roles.decorator.ts
|   |   |-- dto/
|   |   |   `-- pagination.dto.ts
|   |   |-- filters/
|   |   |   `-- all-exceptions.filter.ts
|   |   |-- guards/
|   |   |   |-- jwt-auth.guard.ts
|   |   |   |-- permissions.guard.ts
|   |   |   |-- plan-limit.guard.ts
|   |   |   |-- roles.guard.ts
|   |   |   |-- subscription.guard.ts
|   |   |   `-- tenant.guard.ts
|   |   |-- interceptors/
```

```
|   |   |   |-- audit.interceptor.ts
|   |   |   |-- logging.interceptor.ts
|   |   |   `-- transform-response.interceptor.ts
|   |   |-- middleware/
|   |   |   |-- request-id.middleware.ts
|   |   |   `-- tenant-context.middleware.ts
|   |   `-- utils/
|   |       |-- date.util.ts
|   |       |-- money.util.ts
|   |       |-- sequence.util.ts
|   |       `-- tax.util.ts
|   |-- config/
|   |   |-- app.config.ts
|   |   |-- database.config.ts
|   |   |-- env.validation.ts
|   |   |-- jwt.config.ts
|   |   |-- razorpay.config.ts
|   |   |-- redis.config.ts
|   |   `-- storage.config.ts
|   |-- database/
|   |   |-- prisma.module.ts
|   |   |-- prisma.service.ts
|   |   |-- tenant-prisma.extension.ts
|   |   `-- transaction.helper.ts
|   |-- events/
|   |   |-- bill-settled.listener.ts
|   |   |-- order-placed.listener.ts
|   |   `-- subscription-changed.listener.ts
|   |-- jobs/
|   |   |-- notification.processor.ts
|   |   |-- print.processor.ts
|   |   |-- report-export.processor.ts
|   |   |-- stock-deduct.processor.ts
|   |   `-- subscription-expiry.processor.ts
|   |-- modules/
|   |   |-- admin/
|   |   |   |-- dto/
|   |   |   |-- admin.controller.ts
|   |   |   |-- admin.module.ts
|   |   |   |-- admin.service.ts
|   |   |   |-- impersonation.service.ts
|   |   |   `-- platform-metrics.service.ts
|   |   |-- audit/
|   |   |   |-- dto/
|   |   |   |-- audit.controller.ts
|   |   |   |-- audit.module.ts
|   |   |   `-- audit.service.ts
|   |   |-- auth/
|   |   |   |-- dto/
|   |   |   |   |-- login.dto.ts
|   |   |   |   |-- pin-login.dto.ts
|   |   |   |   `-- refresh.dto.ts
|   |   |   |-- strategies/
|   |   |   |   |-- jwt.strategy.ts
|   |   |   |   `-- refresh.strategy.ts
|   |   |   |-- auth.controller.ts
|   |   |   |-- auth.module.ts
|   |   |   |-- auth.service.ts
|   |   |   |-- pin-login.service.ts
|   |   |   `-- token.service.ts
|   |   |-- billing/
|   |   |   |-- dto/
```

```
|   |   |   |   `-- generate-bill.dto.ts
|   |   |   |-- bill-calculator.service.ts
|   |   |   |-- bill-number.service.ts
|   |   |   |-- billing.controller.ts
|   |   |   |-- billing.module.ts
|   |   |   |-- billing.service.ts
|   |   |   `-- split-bill.service.ts
|   |   |-- customers/
|   |   |   |-- dto/
|   |   |   |-- customers.controller.ts
|   |   |   |-- customers.module.ts
|   |   |   `-- customers.service.ts
|   |   |-- files/
|   |   |   |-- dto/
|   |   |   |-- files.controller.ts
|   |   |   |-- files.module.ts
|   |   |   `-- files.service.ts
|   |   |-- health/
|   |   |   |-- dto/
|   |   |   |-- health.controller.ts
|   |   |   |-- health.module.ts
|   |   |   `-- health.service.ts
|   |   |-- inventory/
|   |   |   |-- dto/
|   |   |   |-- inventory.controller.ts
|   |   |   |-- inventory.module.ts
|   |   |   |-- inventory.service.ts
|   |   |   |-- purchases.service.ts
|   |   |   |-- recipes.service.ts
|   |   |   |-- stock-ledger.service.ts
|   |   |   `-- suppliers.service.ts
|   |   |-- kot/
|   |   |   |-- dto/
|   |   |   |   `-- update-kot-status.dto.ts
|   |   |   |-- kot-number.service.ts
|   |   |   |-- kot-router.service.ts
|   |   |   |-- kot.controller.ts
|   |   |   |-- kot.gateway.ts
|   |   |   |-- kot.module.ts
|   |   |   `-- kot.service.ts
|   |   |-- menu/
|   |   |   |-- dto/
|   |   |   |   `-- create-item.dto.ts
|   |   |   |-- addons.service.ts
|   |   |   |-- categories.service.ts
|   |   |   |-- item-image.service.ts
|   |   |   |-- items.service.ts
|   |   |   |-- menu.controller.ts
|   |   |   |-- menu.module.ts
|   |   |   |-- menu.service.ts
|   |   |   `-- variants.service.ts
|   |   |-- notifications/
|   |   |   |-- dto/
|   |   |   |-- email.provider.ts
|   |   |   |-- notifications.controller.ts
|   |   |   |-- notifications.module.ts
|   |   |   |-- notifications.service.ts
|   |   |   |-- sms.provider.ts
|   |   |   `-- whatsapp.provider.ts
|   |   |-- orders/
|   |   |   |-- dto/
|   |   |   |   |-- add-items.dto.ts
|   |   |   |   `-- create-order.dto.ts
|   |   |   |-- order-number.service.ts
|   |   |   |-- order-state-machine.ts
|   |   |   |-- orders.controller.ts
|   |   |   |-- orders.module.ts
|   |   |   `-- orders.service.ts
|   |   |-- outlets/
|   |   |   |-- dto/
|   |   |   |   `-- create-outlet.dto.ts
|   |   |   |-- outlets.controller.ts
|   |   |   |-- outlets.module.ts
|   |   |   `-- outlets.service.ts
|   |   |-- payments/
|   |   |   |-- dto/
|   |   |   |   `-- settle.dto.ts
|   |   |   |-- payment-gateway.service.ts
|   |   |   |-- payments.controller.ts
|   |   |   |-- payments.module.ts
|   |   |   `-- payments.service.ts
|   |   |-- printing/
|   |   |   |-- dto/
|   |   |   |-- escpos-builder.service.ts
|   |   |   |-- print-agent.gateway.ts
|   |   |   |-- print-job.service.ts
|   |   |   |-- print-template.service.ts
|   |   |   |-- printing.controller.ts
|   |   |   |-- printing.module.ts
|   |   |   `-- printing.service.ts
|   |   |-- realtime/
|   |   |   |-- dto/
|   |   |   |-- realtime.controller.ts
```

```
|   |   |   |-- realtime.events.ts
|   |   |   |-- realtime.gateway.ts
|   |   |   |-- realtime.module.ts
|   |   |   `-- realtime.service.ts
|   |   |-- reports/
|   |   |   |-- dto/
|   |   |   |-- export.service.ts
|   |   |   |-- item-report.service.ts
|   |   |   |-- reports.controller.ts
|   |   |   |-- reports.module.ts
|   |   |   |-- reports.service.ts
|   |   |   |-- sales-report.service.ts
|   |   |   `-- tax-report.service.ts
|   |   |-- roles/
|   |   |   |-- dto/
|   |   |   |-- default-roles.ts
|   |   |   |-- permissions.seed.ts
|   |   |   |-- roles.controller.ts
|   |   |   |-- roles.module.ts
|   |   |   `-- roles.service.ts
|   |   |-- shifts/
|   |   |   |-- dto/
|   |   |   |-- shifts.controller.ts
|   |   |   |-- shifts.module.ts
|   |   |   `-- shifts.service.ts
```

```
|   |   |-- subscriptions/
|   |   |   |-- dto/
|   |   |   |   `-- subscribe.dto.ts
|   |   |   |-- plans.service.ts
|   |   |   |-- razorpay-webhook.controller.ts
|   |   |   |-- subscription-billing.service.ts
|   |   |   |-- subscription-invoice.service.ts
|   |   |   |-- subscription.cron.ts
|   |   |   |-- subscriptions.controller.ts
|   |   |   |-- subscriptions.module.ts
|   |   |   `-- subscriptions.service.ts
|   |   |-- tables/
|   |   |   |-- dto/
|   |   |   |   `-- create-table.dto.ts
|   |   |   |-- areas.service.ts
|   |   |   |-- table-session.service.ts
|   |   |   |-- tables.controller.ts
|   |   |   |-- tables.module.ts
|   |   |   `-- tables.service.ts
|   |   |-- tax/
|   |   |   |-- dto/
|   |   |   |-- gst.service.ts
|   |   |   |-- tax-summary.service.ts
|   |   |   |-- tax.controller.ts
|   |   |   |-- tax.module.ts
|   |   |   `-- tax.service.ts
|   |   |-- tenants/
|   |   |   |-- dto/
|   |   |   |-- tenant-onboarding.service.ts
|   |   |   |-- tenant-settings.service.ts
|   |   |   |-- tenants.controller.ts
|   |   |   |-- tenants.module.ts
|   |   |   `-- tenants.service.ts
|   |   `-- users/
|   |       |-- dto/
|   |       |   |-- create-user.dto.ts
|   |       |   `-- update-user.dto.ts
|   |       |-- users.controller.ts
|   |       |-- users.module.ts
|   |       `-- users.service.ts
|   |-- app.module.ts
|   |-- main.ts
|   `-- worker.ts
|-- test/
|   |-- e2e/
|   |   |-- auth.e2e-spec.ts
|   |   |-- order-to-bill.e2e-spec.ts
|   |   `-- tenant-isolation.e2e-spec.ts
|   |-- unit/
|   |   `-- bill-calculator.spec.ts
|   `-- jest-e2e.json
|-- .env.example
|-- .gitignore
|-- .prettierrc
|-- Dockerfile
|-- README.md
|-- docker-compose.yml
|-- nest-cli.json
|-- package.json
`-- tsconfig.json
```

15. Suggested build order

Step	Deliverable	Why first
1	Foundation: config, Prisma, tenant context, RLS, auth (login, PIN, refresh), RBAC guards, audit	Everything depends on tenant isolation and permissions. Hard to retrofit.
2	Tenant onboarding, outlets, users, roles, areas and tables, menu with image upload, tax groups	Hotel can be set up end to end.
3	Orders and KOT with station routing, realtime gateway, print agent and print jobs	The core value: table to kitchen.
4	Billing calculator, GST bill numbering, payments (cash, UPI, card, split), table free-up	Money path. Needs the most unit tests.
5	Subscriptions with Razorpay, trial, grace, SubscriptionGuard, platform invoices, super admin	Lets you start charging Rs 1000 per month.
6	Reports and day close, shifts, customers, WhatsApp receipts	Retention features for owners.
7	Inventory and recipes, multi-outlet chains, offline PWA hardening	Differentiators vs Petpooja-style competitors.

16. Open decisions for you

- Is Rs 1000 per month inclusive or exclusive of GST, and is there a free trial period?
- Which restaurant types do you target first (dine-in only, or takeaway and delivery too)? This changes tax defaults and flows.
- Will hotels have a Windows PC for the print agent, or Android tablets only?
- Payment gateway for customers paying the hotel (UPI QR on bill) in addition to Razorpay for your subscription.
- Hosting budget and region (AWS Mumbai assumed).
- Mobile approach: PWA first (fastest) or native Android app for waiters.